Task 01 (Part 1):

```
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <ctime>
#include <string>

using namespace std;

const int SIZE = 10;

struct InventoryItem
{     int code;
string name;
double cost;
double price; };

InventoryItem insertItem()
{
    InventoryItem item;
    item.code = 3123 + rand() % 9000;   // auto code

    cout << "Item name: ";
getline(cin >> ws, item.name);
cout << "Item cost: ";     cin >>
item.cost;

    item.price = item.cost * 1.40;      // 40% profit
return item; }

void displayStock(const InventoryItem* item)
{
    cout << left << setw(8) << item->code
         << setw(18) << item->name
         << setw(12) << item->cost
         << setw(12) << item->price << endl;
}

int main()
{
    srand(time(0));
InventoryItem stock[SIZE];

    for (int i = 0; i < SIZE; i++)
stock[i] = insertItem();

    cout << left << setw(8) << "Code"
         << setw(18) << "Name"
         << setw(12) << "Cost"
         << setw(12) << "Price" << endl;
cout << string(50, '-') << endl;

    for (int i = 0; i < SIZE; i++)
displayStock(&stock[i]);     return 0;
}
```

Task 01 (Part 2):

Same as Part 1, but code is a string: first 3 letters (upper case) + _ + running number.

```
#include <iostream>
#include <iomanip>
#include <cctype>
#include <string>
using namespace std;

const int SIZE = 10;
int counter = 31203;   // running integer

struct InventoryItem
{
    string code;       // now a string
string name;     double cost;
double price; };

// builds code like COM_31203
string makeCode(const string& name)
{
    string pre = name.substr(0, 3);     for
(int i = 0; i < (int)pre.size(); i++)
pre[i] = toupper(pre[i]);     return pre +
"_" + to_string(counter++); }

InventoryItem insertItem()
{
    InventoryItem item;     cout
<< "Item name: ";     getline(cin
>> ws, item.name);     cout <<
"Item cost: ";     cin >>
item.cost;

    item.code = makeCode(item.name);
item.price = item.cost * 1.40;
return item; }

void displayStock(const InventoryItem* item)
{
    cout << left << setw(12) << item->code
         << setw(18) << item->name
         << setw(12) << item->cost
         << setw(12) << item->price << endl;
}

int main()
{
    InventoryItem stock[SIZE];
for (int i = 0; i < SIZE; i++)
stock[i] = insertItem();

    cout << left << setw(12) << "Code"
         << setw(18) << "Name"
         << setw(12) << "Cost"
         << setw(12) << "Price" << endl;
cout << string(54, '-') << endl;
```

```
    for (int i = 0; i < SIZE; i++)
displayStock(&stock[i]);     return 0;
}
```

Task 02:

Price per sq ft = Rs. 18. Inches are converted to feet (inches / 12).

```
#include <iostream> using namespace std; const

int PRICE_PER_SQFT = 18;   // Rs. per sq ft

struct FeetInches
{     int feet;
int inches; };

struct RoomDimension
{
    FeetInches length;
    FeetInches Width;
};

// converts feet + inches to feet
double toFeet(const FeetInches& f)
{
    return f.feet + f.inches / 12.0;
}

// cost of carpet for one room
int computeTheCarpetingCost(const RoomDimension& room)
{
    double area = toFeet(room.length) * toFeet(room.Width);
return (int)(area * PRICE_PER_SQFT); }

int main()
{     int n;     cout << "Number
of rooms: ";     cin >> n;

    RoomDimension* rooms = new RoomDimension[n];
for (int i = 0; i < n; i++)
    {
        cout << "Room " << i + 1 << endl;         cout << "
Length (feet inches): ";         cin >> rooms[i].length.feet
>> rooms[i].length.inches;         cout << " Width  (feet
inches): ";
        cin >> rooms[i].Width.feet >> rooms[i].Width.inches;
}

    int choice;
do     {
        cout << "\n1. Total cost of house\n"
             << "2. Cost of each room\n"
             << "0. Exit\n"
<< "Choice: ";         cin
>> choice;         if
(choice == 1)
        {
            int total = 0;             for (int i = 0; i <
n; i++)                 total +=
computeTheCarpetingCost(rooms[i]);             cout <<
"Total cost: Rs. " << total << endl;
        }
        else if (choice == 2)
```

```
        {
            for (int i = 0; i < n; i++)
                cout << "Room " << i + 1 << ": Rs. "
                     << computeTheCarpetingCost(rooms[i]) << endl;
        }
    } while (choice != 0);

    delete[] rooms;
rooms = nullptr;
return 0;
}
```